ETL Process - Step-by-Step Detailed Explanation

Group 7

Team Members:

Gavriel Kirichenko (101119609), Bikash Giri (101575097), Chia-Wei Chang (101570243), Hsi-Teng Liu (101576074), Abdul-Rasaq Omisesan (101571156), Callum Arul (101585383), Friba Hussainyar (101591222), Diparshan Bhattarai (101494737), Sergio Alejandro Medrano Díaz (101446299)

Date: February 17th, 2025

Github link: https://github.com/frogcoder/GBC_Fund_Data_Management/tree/main

Step 1: Creating the Database and Loading Data

1. First, all existing tables in the database are deleted using `DROP TABLE IF EXISTS` to ensure a clean slate.
2. The schema is then created by defining multiple tables such as `categories`, `subcategories`, `employees`, `regions`, `states`, `cities`, `stores`, `ship_modes`, `customer_segments`, `customers`, `products`, `orders`, `order_items`, and `order_returns`.
3. Each table has a primary key to uniquely identify records and foreign keys to establish relationships between tables.
4. The schema is redefined to accommodate specific business requirements, ensuring consistency and referential integrity.
5. Some changes from Step 1 include adjusting column data types, modifying relationships, and adding new constraints.
6. The `order_items` table is updated to include `discount_percentage` and `amount`, which were not present in the initial schema.
7. The `order_items` table remains linked to `orders` and `products` using foreign keys.
8. Once the schema is defined, data from CSV files is loaded into the respective tables using the `LOAD DATA INFILE` command to ensure product names with commas can be properly imported into the database.
9. According to MySQL's default security policy, the data files are assumed to be placed in MySQL's default upload directory: `C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/`.
10. Each `LOAD DATA INFILE` statement specifies:
    - The file location.
    - Fields are separated by commas and optionally enclosed in double quotes.
    - The columns to be imported.

This step ensures that the database is properly structured and populated with initial data.

Step 1: SQL Code

```sql
DROP TABLE IF EXISTS order_returns;
DROP TABLE IF EXISTS order_items;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS customers;
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS subcategories;
DROP TABLE IF EXISTS categories;
DROP TABLE IF EXISTS stores;
DROP TABLE IF EXISTS cities;
DROP TABLE IF EXISTS states;
DROP TABLE IF EXISTS regions;
DROP TABLE IF EXISTS employees;
DROP TABLE IF EXISTS ship_modes;
DROP TABLE IF EXISTS customer_segments;


CREATE TABLE categories (
    category_code CHAR(3) NOT NULL,
    category_name VARCHAR(64) NOT NULL,
    PRIMARY KEY (category_code)
);

CREATE TABLE subcategories (
    category_code CHAR(3) NOT NULL,
    subcategory_code CHAR(2) NOT NULL,
    subcategory_name VARCHAR(64) NOT NULL,
    PRIMARY KEY (category_code, subcategory_code),
    FOREIGN KEY (category_code) REFERENCES categories(category_code)
);

CREATE TABLE employees (
    employee_id CHAR(8),
    employee_name VARCHAR(64),
    PRIMARY KEY (employee_id)
);

CREATE TABLE regions (
    region_code CHAR(1) NOT NULL,
    region_name VARCHAR(16) NOT NULL,
    manager_employee_id CHAR(8) NOT NULL,
```

```
    PRIMARY KEY (region_code),
    FOREIGN KEY (manager_employee_id) REFERENCES employees(employee_id)
);

CREATE TABLE states (
    state_code CHAR(2) NOT NULL,
    region_code CHAR(1) NOT NULL,
    state_name VARCHAR(24) NOT NULL,
    PRIMARY KEY (state_code),
    FOREIGN KEY (region_code) REFERENCES regions(region_code)
);

CREATE TABLE cities (
    city_id INT NOT NULL,
    state_code CHAR(2) NOT NULL,
    city_name VARCHAR(64) NOT NULL,
    PRIMARY KEY (city_id),
    FOREIGN KEY (state_code) REFERENCES states(state_code)
);

CREATE TABLE stores (
    postal_code INT NOT NULL,
    city_id INT NOT NULL,
    PRIMARY KEY (postal_code),
    FOREIGN KEY (city_id) REFERENCES cities(city_id)
);

CREATE TABLE ship_modes (
    ship_mode_code CHAR(1) NOT NULL,
    ship_mode_name VARCHAR(32) NOT NULL,
    PRIMARY KEY (ship_mode_code)
);

CREATE TABLE customer_segments (
    segment_code CHAR(1) NOT NULL,
    segment_name VARCHAR(32) NOT NULL,
    PRIMARY KEY (segment_code)
);

CREATE TABLE customers (
    customer_id CHAR(8) NOT NULL,
    segment_code CHAR(1) NOT NULL,
```

```
    customer_name VARCHAR(32) NOT NULL,
    PRIMARY KEY (customer_id),
    FOREIGN KEY (segment_code) REFERENCES customer_segments(segment_code)
);

CREATE TABLE products (
    product_id CHAR(15) NOT NULL,
    product_name VARCHAR(256) NOT NULL,
    category_code CHAR(3) NOT NULL,
    subcategory_code CHAR(2) NOT NULL,
    unit_price DECIMAL(7, 2) NOT NULL,
    unit_cost DECIMAL(7, 2) NOT NULL,
    PRIMARY KEY (product_id, product_name),
    FOREIGN KEY (category_code, subcategory_code) REFERENCES
subcategories(category_code, subcategory_code)
);

CREATE TABLE orders (
    order_id CHAR(14) NOT NULL,
    customer_id CHAR(8) NOT NULL,
    postal_code INT NOT NULL,
    order_date DATE NOT NULL,
    ship_mode_code CHAR(1) NULL,
    shipping_date DATE NULL,
    PRIMARY KEY (order_id),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (postal_code) REFERENCES stores(postal_code),
    FOREIGN KEY (ship_mode_code) REFERENCES ship_modes(ship_mode_code)
);

CREATE TABLE order_items (
    order_item_id INT NOT NULL,
    order_id CHAR(14) NOT NULL,
    product_id CHAR(15) NOT NULL,
    product_name VARCHAR(256) NOT NULL,
    unit_price DECIMAL(7, 2) NOT NULL,
    unit_cost DECIMAL(7, 2) NOT NULL,
    quantity INT NOT NULL,
    discount_percentage DECIMAL(2, 2) NOT NULL,
    amount DECIMAL(8, 2) NOT NULL,
    PRIMARY KEY (order_item_id),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
```

```
    FOREIGN KEY (product_id, product_name) REFERENCES products(product_id,
product_name)
);

CREATE TABLE order_returns (
    order_id CHAR(14) NOT NULL,
    PRIMARY KEY (order_id)
);



LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server
8.0/Uploads/categories.csv"
INTO TABLE categories
Fields TERMINATED BY ','
       OPTIONALLY ENCLOSED BY '"'
       ESCAPED BY '"'
IGNORE 1 LINES
(category_code, category_name);

LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server
8.0/Uploads/subcategories.csv"
INTO TABLE subcategories
Fields TERMINATED BY ','
       OPTIONALLY ENCLOSED BY '"'
       ESCAPED BY '"'
IGNORE 1 LINES
(category_code, subcategory_code, subcategory_name);

LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/products.csv"
INTO TABLE products
Fields TERMINATED BY ','
       OPTIONALLY ENCLOSED BY '"'
       ESCAPED BY '"'
IGNORE 1 LINES
(product_id, product_name, category_code, subcategory_code, unit_price,
unit_cost);

LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server
8.0/Uploads/customer_segments.csv"
INTO TABLE customer_segments
Fields TERMINATED BY ','
```

```
       OPTIONALLY ENCLOSED BY '"'
       ESCAPED BY '"'
IGNORE 1 LINES
(segment_code, segment_name);

LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server
8.0/Uploads/ship_modes.csv"
INTO TABLE ship_modes
Fields TERMINATED BY ','
       OPTIONALLY ENCLOSED BY '"'
       ESCAPED BY '"'
IGNORE 1 LINES
(ship_mode_code, ship_mode_name);

LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/employees.csv"
INTO TABLE employees
Fields TERMINATED BY ','
       OPTIONALLY ENCLOSED BY '"'
       ESCAPED BY '"'
IGNORE 1 LINES
(employee_id, employee_name);

LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/regions.csv"
INTO TABLE regions
Fields TERMINATED BY ','
       OPTIONALLY ENCLOSED BY '"'
       ESCAPED BY '"'
IGNORE 1 LINES
(region_code, region_name, manager_employee_id);

LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/states.csv"
INTO TABLE states
Fields TERMINATED BY ','
       OPTIONALLY ENCLOSED BY '"'
       ESCAPED BY '"'
IGNORE 1 LINES
(state_code, region_code, state_name);

LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/cities.csv"
INTO TABLE cities
Fields TERMINATED BY ','
       OPTIONALLY ENCLOSED BY '"'
```

```
       ESCAPED BY '"'
IGNORE 1 LINES
(city_id, state_code, city_name);

LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/stores.csv"
INTO TABLE stores
Fields TERMINATED BY ','
       OPTIONALLY ENCLOSED BY '"'
       ESCAPED BY '"'
IGNORE 1 LINES
(postal_code, city_id);

LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/customers.csv"
INTO TABLE customers
Fields TERMINATED BY ','
       OPTIONALLY ENCLOSED BY '"'
       ESCAPED BY '"'
IGNORE 1 LINES
(customer_id, segment_code, customer_name);

LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/orders.csv"
INTO TABLE orders
Fields TERMINATED BY ','
       OPTIONALLY ENCLOSED BY '"'
       ESCAPED BY '"'
IGNORE 1 LINES
(order_id, customer_id, postal_code, order_date, ship_mode_code,
shipping_date);

LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server
8.0/Uploads/order_items.csv"
INTO TABLE order_items
Fields TERMINATED BY ','
       OPTIONALLY ENCLOSED BY '"'
       ESCAPED BY '"'
IGNORE 1 LINES
(order_item_id, order_id, product_id, product_name, unit_price, unit_cost,
quantity, discount_percentage, amount);

LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server
8.0/Uploads/order_returns.csv"
INTO TABLE order_returns
```

```
Fields TERMINATED BY ','
       OPTIONALLY ENCLOSED BY '"'
       ESCAPED BY '"'
IGNORE 1 LINES;
```

Step 2: Generating Executive Reports

1. The executive report tables (`report_current_month`, `report_prev_month`, `report_prev_year`) are created to store sales performance data.
2. The report queries aggregate data by region and state, calculating:
    - Total quantity sold (`SUM(i.quantity)`)
    - Total revenue (`SUM(i.amount)`)
    - Total profit (`SUM(i.amount) - SUM(i.unit_cost * i.quantity)`)
3. The time filters ensure that the reports fetch data for:
    - The current month (`2021-11-01` to `2021-12-01`)
    - The previous month (`2021-10-01` to `2021-11-01`)
    - The same month in the previous year (`2020-11-01` to `2020-12-01`)
4. A final report compares the current month's revenue with the previous month and previous year to measure growth trends.
5. The report includes calculations for:
    - Month-over-month revenue growth
    - Year-over-year revenue growth

This step enables business leaders to evaluate sales performance and identify trends.

Step 2: SQL Code

```
DROP TABLE IF EXISTS report_current_month;
CREATE TABLE report_current_month
SELECT r.region_name,
       s.state_name,
       SUM(i.quantity) AS monthly_quantity_sold,
       SUM(i.amount) - SUM(i.unit_cost * i.quantity) AS monthly_profit,
       SUM(i.amount) AS monthly_revenue
FROM orders o
     JOIN order_items i ON o.order_id = i.order_id
     JOIN stores ON o.postal_code = stores.postal_code
     JOIN cities c ON stores.city_id = c.city_id
     JOIN states s ON c.state_code = s.state_code
     JOIN regions r ON s.region_code = r.region_code
WHERE o.order_date >= '2021-11-01' AND o.order_date < '2021-12-01'
GROUP BY YEAR(o.order_date),
```

```
        MONTH(o.order_date),
        r.region_name,
        s.state_name;

DROP TABLE IF EXISTS report_prev_month;
CREATE TABLE report_prev_month
SELECT r.region_name,
       s.state_name,
       SUM(i.quantity) AS monthly_quantity_sold,
       SUM(i.amount) - SUM(i.unit_cost * i.quantity) AS monthly_profit,
       SUM(i.amount) AS monthly_revenue
FROM orders o
     JOIN order_items i ON o.order_id = i.order_id
     JOIN stores ON o.postal_code = stores.postal_code
     JOIN cities c ON stores.city_id = c.city_id
     JOIN states s ON c.state_code = s.state_code
     JOIN regions r ON s.region_code = r.region_code
WHERE o.order_date >= '2021-10-01' AND o.order_date < '2021-11-01'
GROUP BY r.region_name,
         s.state_name;

DROP TABLE IF EXISTS report_prev_year;
CREATE TABLE report_prev_year
SELECT r.region_name,
       s.state_name,
       SUM(i.quantity) AS monthly_quantity_sold,
       SUM(i.amount) - SUM(i.unit_cost * i.quantity) AS monthly_profit,
       SUM(i.amount) AS monthly_revenue
FROM orders o
     JOIN order_items i ON o.order_id = i.order_id
     JOIN stores ON o.postal_code = stores.postal_code
     JOIN cities c ON stores.city_id = c.city_id
     JOIN states s ON c.state_code = s.state_code
     JOIN regions r ON s.region_code = r.region_code
WHERE o.order_date >= '2020-11-01' AND o.order_date < '2020-12-01'
GROUP BY r.region_name,
         s.state_name;


SELECT c.region_name, c.state_name,
       c.monthly_quantity_sold, c.monthly_profit, c.monthly_revenue,
       m.monthly_revenue AS prev_monthly_revenue,
```

```
        c.monthly_revenue - m.monthly_revenue AS monthly_revenue_growth,
        (c.monthly_revenue - m.monthly_revenue) / m.monthly_revenue AS
monthly_revenue_growth_rate,
        y.monthly_revenue AS prev_year_monthly_revenue,
        c.monthly_revenue - y.monthly_revenue AS
year_over_year_monthy_revenue_growth,
        (c.monthly_revenue - y.monthly_revenue) / y.monthly_revenue AS
year_over_year_monthy_revenue_growth_rate
FROM report_current_month c
LEFT JOIN report_prev_month m ON c.region_name=m.region_name AND
c.state_name=m.state_name
LEFT JOIN report_prev_year y on c.region_name=y.region_name AND
c.state_name=y.state_name
ORDER BY region_name, state_name;
```

Step 3: Generating Operational Reports

1. The operational report provides weekly sales data, helping business managers analyze performance at a granular level.
2. The query aggregates sales data by region, state, city, and product, calculating:
    - Total quantity sold in the week (`SUM(oi.quantity)`)
    - Weekly revenue (`SUM(oi.amount)`)
    - Weekly profit (`SUM(oi.amount) - SUM(oi.unit_cost * oi.quantity)`)
    - Discount applied (`1 - (SUM(oi.amount) / SUM(oi.unit_price * oi.quantity))`)
3. The date filter (`2021-12-19` to `2021-12-26`) ensures that only a specific week's data is analyzed.
4. The results are sorted by region, state, city, and product name for easier readability.

This step helps in monitoring operational performance, detecting sales patterns, and making inventory decisions.

Step 3: SQL Code

```
SELECT
    r.region_name AS Region,
    s.state_name AS State,
    c.city_name AS City,
    p.product_name AS Item_name,
    oi.unit_price AS Unit_Price,
```

```
    SUM(oi.quantity) AS Weekly_Quantity_Sold,
    SUM(oi.amount) AS Weekly_Revenue_Sales,
    SUM(oi.amount) - SUM(oi.unit_cost * oi.quantity) AS Weekly_Profit,
    1 - (SUM(oi.amount) / SUM(oi.unit_price * oi.quantity)) AS
Weekly_Discount_Applied

FROM
    orders o
JOIN
    stores st ON o.postal_code = st.postal_code
JOIN
    cities c ON st.city_id = c.city_id
JOIN
    states s ON c.state_code = s.state_code
JOIN
    regions r ON s.region_code = r.region_code
JOIN
    order_items oi ON o.order_id = oi.order_id
JOIN
    products p ON oi.product_id = p.product_id

WHERE
    o.order_date >= '2021-12-19'
    AND o.order_date < '2021-12-26'

GROUP BY
    r.region_name, s.state_name, c.city_name, p.product_name, oi.unit_price

ORDER BY
    r.region_name, s.state_name, c.city_name, p.product_name
```